

S1. Tarea. Simulador de planificación de procesos

Estudiantes:

Juan Pablo Correa Rave

Juan Sebastián Briñez Orjuela

Simón Madrid Tamayo

Tecnología en Desarrollo de Software, Institución Universitaria Pascua Bravo
sistemas operativos
Grupo 100
Yexid Montenegro

Medellín

INTRODUCCIÓN

- Deberás implementar un simulador que represente al menos tres procesos del SIGET, cada uno con distintos atributos (por ejemplo, tiempo_irrupcion, prioridad_alerta, tamaño_datos).
- Deberás aplicar un mínimo de dos algoritmos de planificación de procesos, como FIFO, SJF, round-robin o prioridad.
- La solución debe permitir observar claramente los estados por los que pasa cada proceso (Nuevo, Listo, En ejecución, Bloqueado y Terminado) y cómo se gestionan en ejecución.
- El desarrollo puede realizarse en un lenguaje de programación visto en clase como C++, Java, Arduino o Python.

OBJETIVOS:

La simulación debe ser una solución práctica que simule el funcionamiento de un planificador de procesos, para lo que debes entregar:

- El archivo fuente del código (en .cpp, .java, .py, .ino u otro según el entorno).
- Evidencia de ejecución (video corto) que demuestre el funcionamiento del simulador.
- Un documento explicativo breve (informe o relatoría de máximo dos páginas) que incluya:
 - a. Los objetivos y las decisiones de diseño.
 - b. Los algoritmos utilizados y las observaciones.
 - c. Las conclusiones sobre el desempeño de los algoritmos en el contexto del SIGET.

EXPLICACIÓN DEL CÓDIGO:

Estructura del Semáforo y variables de control.

```
#include <iostream>
#include <string>

using namespace std;

// -----
// Estructura que representa un proceso del sistema SIGET
// -----
struct Proceso
{
    int id;
    string nombre;

    // Atributos del proceso
    int tiempoEjecucion;
    int prioridadAlerta;
    int tamanoDatos;

    // Estado actual del proceso
    string estado;
};
```

En este primer bloque se importan las librerías `iostream` y `string`, necesarias para mostrar información en pantalla y trabajar con textos. A continuación, se define la estructura `Proceso`, donde se almacenan los datos de cada proceso del SIGET, como su identificación, nombre, tiempo de ejecución, prioridad de alerta, tamaño de datos y estado actual.

```
// -----  
// Funcion para mostrar la informacion de un proceso  
// -----  
void mostrarProceso(Proceso p)  
{  
    cout << "Proceso: " << p.nombre << endl;  
    cout << "ID: " << p.id << endl;  
    cout << "Tiempo de ejecucion: " << p.tiempoEjecucion << " segundos" << endl;  
    cout << "Prioridad de alerta: " << p.prioridadAlerta << endl;  
    cout << "Tamano de datos: " << p.tamanoDatos << " MB" << endl;  
    cout << "Estado: " << p.estado << endl;  
}
```

```
// ALGORITMO 1: FIFO
//
// FIFO significa "First In, First Out".
// El primer proceso que llega es el primero en ejecutarse.
// -----
void ejecutarFIFO(Proceso procesos[], int cantidad)
{
    cout << "\n===== " << endl;
    cout << "      ALGORITMO FIFO " << endl;
    cout << "===== \n" << endl;

    for (int i = 0; i < cantidad; i++)
    {
        // El proceso pasa de Nuevo a Listo
        procesos[i].estado = "Listo";

        cout << procesos[i].nombre
            << " -> Nuevo -> Listo " << endl;

        // El proceso comienza a ejecutarse
        procesos[i].estado = "En ejecucion";

        cout << procesos[i].nombre
            << " -> En ejecucion " << endl;

        // Simulamos un bloqueo durante el procesamiento
        procesos[i].estado = "Bloqueado";

        cout << procesos[i].nombre
            << " -> Bloqueado " << endl;

        // Finalmente termina
        procesos[i].estado = "Terminado";

        cout << procesos[i].nombre
            << " -> Terminado \n" << endl;
    }
}
```

En este bloque se implementa el algoritmo FIFO, donde los procesos se ejecutan en el mismo orden en que fueron registrados. Mediante un ciclo `for`, cada proceso cambia sucesivamente por los estados Listo, En ejecución, Bloqueado y Terminado, mostrando en pantalla cada transición para visualizar cómo se gestiona durante la simulación.

```
// -----
// ALGORITMO 2: PRIORIDAD
//
// El proceso con mayor prioridad se ejecuta primero.
// Una prioridad de 10 es mayor que una prioridad de 5.
// -----
void ejecutarPrioridad(Proceso procesos[], int cantidad)
{
    cout << "\n===== " << endl;
    cout << "      ALGORITMO POR PRIORIDAD" << endl;
    cout << "===== \n" << endl;

    // Ordenamos los procesos de mayor a menor prioridad.
    // Utilizamos un ordenamiento sencillo tipo burbuja.
    for (int i = 0; i < cantidad - 1; i++)
    {
        for (int j = 0; j < cantidad - 1 - i; j++)
        {
            if (procesos[j].prioridadAlerta <
                procesos[j + 1].prioridadAlerta)
            {
                Proceso temporal = procesos[j];
                procesos[j] = procesos[j + 1];
                procesos[j + 1] = temporal;
            }
        }
    }
}
```

En este bloque se implementa el algoritmo de planificación por prioridad, donde los procesos con mayor nivel de alerta son atendidos primero. Para esto, se utiliza un ordenamiento tipo burbuja que compara las prioridades y reorganiza los procesos de mayor a menor, permitiendo establecer el orden en que serán ejecutados.

```
// Ejecutamos los procesos en el nuevo orden
for (int i = 0; i < cantidad; i++)
{
    procesos[i].estado = "Listo";

    cout << procesos[i].nombre
         << " -> Nuevo -> Listo" << endl;

    procesos[i].estado = "En ejecucion";

    cout << procesos[i].nombre
         << " -> En ejecucion" << endl;

    procesos[i].estado = "Bloqueado";

    cout << procesos[i].nombre
         << " -> Bloqueado" << endl;

    procesos[i].estado = "Terminado";

    cout << procesos[i].nombre
         << " -> Terminado\n" << endl;
}
```

En este bloque se ejecutan los procesos siguiendo el orden establecido por sus prioridades. Cada proceso pasa por los estados Listo, En ejecución, Bloqueado y Terminado, mostrando en pantalla cada cambio para visualizar el ciclo de ejecución del proceso.

```
// PROGRAMA PRINCIPAL
// -----
int main()
{
    // Creamos 4 procesos relacionados con el SIGET.
    Proceso procesos[4] =
    {
        {1, "Procesamiento de trafico", 5, 4, 500, "Nuevo"},
        {2, "Alerta de accidente", 3, 10, 100, "Nuevo"},
        {3, "Estado de semaforos", 4, 8, 250, "Nuevo"},
        {4, "Analisis de datos", 7, 2, 800, "Nuevo"}
    };

    // -----
    // Mostrar los procesos inicialmente
    // -----
    cout << "===== " << endl;
    cout << "      SIMULADOR DE CPU - SIGET" << endl;
    cout << "===== \n" << endl;

    cout << "Procesos registrados:\n" << endl;

    for (int i = 0; i < 4; i++)
    {
        mostrarProceso(procesos[i]);
        cout << "-----" << endl;
    }

    // -----
    // Ejecutamos FIFO
    // -----
    ejecutarFIFO(procesos, 4);

    // -----
    // Volvemos a crear los procesos para realizar
    // una nueva simulacion utilizando prioridad.
    // -----
}
```

```

// -----
Proceso procesosPrioridad[4] =
{
    {1, "Procesamiento de trafico", 5, 4, 500, "Nuevo"},
    {2, "Alerta de accidente", 3, 10, 100, "Nuevo"},
    {3, "Estado de semaforos", 4, 8, 250, "Nuevo"},
    {4, "Análisis de datos", 7, 2, 800, "Nuevo"}
};

// -----
// Ejecutamos el algoritmo por prioridad
// -----
ejecutarPrioridad(procesosPrioridad, 4);

// -----
// Final del programa
// -----
cout << "===== " << endl;
cout << "      SIMULACION FINALIZADA " << endl;
cout << "===== " << endl;

return 0;
}

```

En este bloque se inicia el programa principal y se crean cuatro procesos del SIGET con sus respectivos atributos. Luego, se muestra la información inicial de cada proceso y se ejecuta el algoritmo FIFO. Posteriormente, se crean nuevamente los procesos para reiniciar su estado y realizar una segunda simulación utilizando el algoritmo de prioridad. Finalmente, se muestra el mensaje indicando que la simulación ha terminado.

RESULTADOS EN PANTALLA:

C:\Users\Usuario\Desktop\Pascual Bravo\3. Sistemas Operativos\S1. Tarea. Simulador de plar

```
=====
SIMULADOR DE CPU - SIGET
=====

Procesos registrados:

Proceso: Procesamiento de trafico
ID: 1
Tiempo de ejecucion: 5 segundos
Prioridad de alerta: 4
Tamano de datos: 500 MB
Estado: Nuevo
-----

Proceso: Alerta de accidente
ID: 2
Tiempo de ejecucion: 3 segundos
Prioridad de alerta: 10
Tamano de datos: 100 MB
Estado: Nuevo
-----

Proceso: Estado de semaforos
ID: 3
Tiempo de ejecucion: 4 segundos
Prioridad de alerta: 8
Tamano de datos: 250 MB
Estado: Nuevo
-----

Proceso: Analisis de datos
ID: 4
Tiempo de ejecucion: 7 segundos
Prioridad de alerta: 2
Tamano de datos: 800 MB
Estado: Nuevo
-----
```

Este bloque corresponde a la salida inicial del programa, donde se muestra la información de los cuatro procesos registrados en el SIGET. Para cada proceso se presentan sus atributos principales, como nombre, ID, tiempo de ejecución, prioridad de alerta, tamaño de datos y estado inicial, permitiendo verificar las características de cada proceso antes de aplicar los algoritmos de planificación.

```
=====
ALGORITMO FIFO
=====

Procesamiento de trafico -> Nuevo -> Listo
Procesamiento de trafico -> En ejecucion
Procesamiento de trafico -> Bloqueado
Procesamiento de trafico -> Terminado

Alerta de accidente -> Nuevo -> Listo
Alerta de accidente -> En ejecucion
Alerta de accidente -> Bloqueado
Alerta de accidente -> Terminado

Estado de semaforos -> Nuevo -> Listo
Estado de semaforos -> En ejecucion
Estado de semaforos -> Bloqueado
Estado de semaforos -> Terminado

Analisis de datos -> Nuevo -> Listo
Analisis de datos -> En ejecucion
Analisis de datos -> Bloqueado
Analisis de datos -> Terminado
```

En este bloque se muestra la ejecución del algoritmo FIFO, donde los procesos son atendidos en el mismo orden en que fueron registrados. Cada proceso pasa de Nuevo a Listo, luego a En ejecución, después a Bloqueado y finalmente a Terminado, permitiendo visualizar de forma secuencial cómo se gestionan los procesos durante la simulación.

```
=====
ALGORITMO POR PRIORIDAD
=====

Alerta de accidente -> Nuevo -> Listo
Alerta de accidente -> En ejecucion
Alerta de accidente -> Bloqueado
Alerta de accidente -> Terminado

Estado de semaforos -> Nuevo -> Listo
Estado de semaforos -> En ejecucion
Estado de semaforos -> Bloqueado
Estado de semaforos -> Terminado

Procesamiento de trafico -> Nuevo -> Listo
Procesamiento de trafico -> En ejecucion
Procesamiento de trafico -> Bloqueado
Procesamiento de trafico -> Terminado

Analisis de datos -> Nuevo -> Listo
Analisis de datos -> En ejecucion
Analisis de datos -> Bloqueado
Analisis de datos -> Terminado

=====
SIMULACION FINALIZADA
=====
```

En este bloque se muestra la ejecución del algoritmo por prioridad, donde los procesos se organizan de mayor a menor nivel de prioridad de alerta. Por esta razón, la alerta de accidente se ejecuta primero, seguida del estado de semáforos, el procesamiento de tráfico y finalmente el análisis de datos. Cada proceso pasa por los estados Nuevo, Listo, En ejecución, Bloqueado y Terminado, y al final se indica que la simulación ha finalizado.